Claude Certified Developer – Foundations (CCDV-F)

Gabarito comentado · Dia 1 · Simulado 3

53 questões · Tempo: 120 minutos · Português

Resumo do gabarito

#	Resposta correta	Domínio	Skill	Nível
1	**A, B, C**	D2	Software Engineering Foundations	difícil
2	**A, B**	D8	Tool Implementation	média
3	**C**	D5	LLM Fundamentals	fácil
4	**B**	D1	Agent Patterns & Frameworks	média
5	**A, B**	D3	Claude Code	média
6	**A**	D5	Technical Fundamentals	fácil
7	**C, D**	D2	Claude Application Design	média
8	**A**	D8	Agentic Customization	fácil
9	**B, C**	D1	Agent Construction with Claude	média
10	**A**	D2	Understanding Requirements	fácil
11	**D, E**	D3	Claude Code	média
12	**B, D, E**	D5	Cost and Token Management	difícil
13	**A, B**	D2	Claude Application Design	média
14	**A, B, C**	D6	Context Engineering	difícil
15	**B, E**	D7	Identity, Secrets and Key Management	fácil
16	**B**	D6	Prompt Engineering	média
17	**C**	D7	AI Application Security	fácil
18	**A, D**	D8	Agentic Customization	média
19	**A, E**	D6	Output Handling	média
20	**B, D**	D2	Understanding Requirements	média
21	**C**	D2	Configuration Management	difícil
22	**C**	D8	MCP Server Development	difícil
23	**B**	D5	Cost and Token Management	média
24	**D**	D2	Systems Life Cycle	difícil
25	**A, D**	D1	Agent Patterns & Frameworks	média
26	**B**	D5	LLM Fundamentals	difícil
27	**C**	D7	Claude Hooks	média
28	**B, D**	D8	Tool Implementation	difícil
29	**D**	D4	Debugging & Error Handling	difícil
30	**B**	D8	Tool Implementation	média
31	**B**	D2	Claude API Mechanics	média
32	**A**	D1	Agent Architecture	fácil
33	**A, C**	D5	LLM Fundamentals	média
34	**A, C**	D5	Technical Fundamentals	média
35	**B**	D1	Agent Patterns & Frameworks	fácil
36	**A**	D5	Technical Fundamentals	média
37	**A, D, E**	D2	Software Engineering Foundations	média
38	**A**	D1	Agent Construction with Claude	fácil
39	**B**	D2	Claude Application Design	fácil

#	Resposta correta	Domínio	Skill	Nível
40	**C**	D5	Model Selection and Tradeoffs	fácil
41	**D**	D2	Software Engineering Foundations	fácil
42	**A**	D2	Claude Application Design	fácil
43	**A, C, D**	D2	Claude API Mechanics	média
44	**D**	D2	Claude API Mechanics	fácil
45	**A**	D6	Output Handling	fácil
46	**A**	D6	Context Engineering	média
47	**D**	D2	Software Engineering Foundations	fácil
48	**C**	D2	Configuration Management	fácil
49	**D**	D6	Prompt Engineering	fácil
50	**A, C, E**	D7	AI Application Security	difícil
51	**D**	D2	Claude Application Design	fácil
52	**C**	D1	Agent Architecture	fácil
53	**D**	D1	Agent Construction with Claude	difícil

Gabarito comentado

Questão 1 · D2 Applications & Integration · Software Engineering Foundations · difícil

O CI de uma equipe chama a API real do Claude em todo pull request: o pipeline é lento, caro e instável. A equipe quer testar o código de integração de forma confiável. Selecione 3 práticas adequadas.

- **A.** **Usar, no pipeline do PR, dublês do cliente da API com respostas realistas gravadas, incluindo tool_use, erros 429/529 e eventos de erro no stream** [CORRETA]
- **B.** **Medir a qualidade das respostas do modelo em uma suíte de evals separada, pois dublês só verificam o código de integração e não o comportamento do modelo** [CORRETA]
- **C.** **Rodar periodicamente uma pequena suíte de contrato contra a API real para checar se os dublês ainda refletem o formato real, abrindo tarefa de ajuste se divergirem** [CORRETA]
- **D.** Escrever dublês apenas com o caminho feliz em JSON feito à mão, já que nunca será necessário comparar com respostas reais
- **E.** Manter as chamadas reais em todo PR e repetir o job até ficar verde, pois a instabilidade é só ruído

Resposta correta: A, B, C. Corretas (A, C e B): dublês realistas tornam os testes de PR rápidos e determinísticos; testes de contrato periódicos detectam divergência entre dublê e serviço sem bloquear PRs não relacionados; e a qualidade do modelo é verificada por evals, não por mocks. D está errada porque dublês feitos à mão sem verificação se afastam da realidade e passam a dar falsa confiança. E está errada porque repetir até passar esconde falhas, mantém custo e lentidão e normaliza testes instáveis.
Fonte: https://martinfowler.com/bliki/ContractTest.html

Questão 2 · D8 Tools & MCPs · Tool Implementation · média

Sua tool update_ticket tem o parâmetro status como string livre. Em produção Claude envia "Done", "finished" e "fechado", mas a API só aceita open, in_progress e closed. Quais mudanças na definição da tool atacam a causa? Selecione 2.

- **A.** **Declarar status com "enum": ["open", "in_progress", "closed"] no input_schema e explicar cada valor na description do parâmetro.** [CORRETA]
- **B.** **Adicionar input_examples com chamadas válidas, mostrando os valores de status aceitos.** [CORRETA]
- **C.** Usar tool_choice forçado do tipo tool, para que Claude sempre chame update_ticket.
- **D.** Reduzir a description da tool a uma frase curta, para não confundir o modelo.
- **E.** Mapear em silêncio, no handler, qualquer valor desconhecido para "open", para nunca falhar.

Resposta correta: A, B. Corretas (A e B): enum com descrições fecha o conjunto de valores, e input_examples (que precisam ser válidos segundo o input_schema) mostram o uso correto. E esconde o erro e corrompe dados. C obriga a chamar a tool, mas não restringe os valores do parâmetro. D vai contra a boa prática: descrições detalhadas são o fator mais importante no desempenho da tool.
Fonte: https://platform.claude.com/docs/en/agents-and-tools/tool-use/define-tools

Questão 3 · D5 Model Selection & Optimization · LLM Fundamentals · fácil

Um dev ativa o thinking com display "omitted" para a resposta não trazer texto de raciocínio, achando que assim deixará de pagar pelo thinking. O que de fato acontece na cobrança?

- **A.** O raciocínio é cobrado como tokens de input, a uma taxa menor que a de output.
- **B.** Só o texto visível é cobrado; como o campo thinking vem vazio, os tokens de raciocínio ficam sem custo.
- **C.** **Os tokens de thinking realmente gerados são cobrados como output, independentemente do display; o detalhamento aparece em usage.output_tokens_details.thinking_tokens.** [CORRETA]
- **D.** É cobrado apenas um resumo do raciocínio, proporcional ao tamanho do texto resumido.

Resposta correta: C. Correta (C): o display muda só o que é devolvido; o raciocínio gerado é faturado como output e output_tokens é o total autoritativo. B está errada porque omitir não zera o custo. A está errada porque thinking gerado é output, não input. D está errada porque a cobrança reflete o raciocínio completo, não o resumo visível.
Fonte: https://platform.claude.com/docs/en/build-with-claude/thinking-steering-and-cost

Questão 4 · D1 Agents & Workflows · Agent Patterns & Frameworks · média

Um agente do Agent SDK tem três ferramentas customizadas, todas apenas de consulta (lookup_customer, lookup_order, lookup_inventory). Quando Claude pede as três no mesmo turno, elas rodam uma após a outra e a latência triplica. Qual a melhor correção?

A. Adicionar as três ferramentas a allowed_tools, para que o SDK as trate como seguras para execução concorrente.

B. **Marcar as ferramentas com readOnlyHint nas annotations; customizadas são sequenciais por padrão.** [CORRETA]

C. Reduzir effort para low, para que Claude peça menos chamadas por turno e conclua o conjunto mais rápido.

D. Aumentar max_turns, para que Claude possa distribuir as chamadas por vários turnos e peça menos delas de cada vez.

Resposta correta: B. Correta (B): ferramentas somente leitura podem rodar concorrentemente, enquanto as que modificam estado rodam em sequência; as customizadas são sequenciais por padrão, e readOnlyHint nas annotations habilita o paralelismo. A só aprova as chamadas sem prompt de permissão; não muda a concorrência. D não reduz a latência e apenas permite mais turnos. C altera a profundidade de raciocínio, não o modo de execução das ferramentas.
Fonte: https://code.claude.com/docs/en/agent-sdk/agent-loop

Questão 5 · D3 Claude Code · Claude Code · média

No meio de uma conversa longa no Claude Code, o desenvolvedor quer testar uma abordagem alternativa de refatoração partindo do histórico atual, sem alterar a conversa original, que deve continuar disponível. Selecione 2 formas adequadas.

A. **/branch, com um nome opcional para a nova conversa.** [CORRETA]

B. **claude --continue --fork-session, que copia o histórico para um novo ID de sessão.** [CORRETA]

C. /compact, que resume o histórico para liberar espaço.

D. /clear, que começa com o contexto vazio e preserva o histórico anterior.

E. claude --resume <nome>, que reabre a sessão e segue a partir dela.

Resposta correta: A, B. Corretas (A e B): /branch e --fork-session copiam o histórico para um novo ID e deixam a original intacta. C reescreve o histórico da própria sessão como resumo, sem criar ramificação. D salva a sessão anterior, mas a nova conversa não herda o histórico, então não parte do ponto atual. E reabre a sessão com o mesmo ID e acrescenta mensagens à conversa existente, alterando a original.
Fonte: https://code.claude.com/docs/en/sessions

Questão 6 · D5 Model Selection & Optimization · Technical Fundamentals · fácil

Um serviço captura Exception genérica ao chamar a API e decide o que fazer com checagens como `"overloaded" in str(e)`. Após uma mudança de redação nas mensagens de erro, o tratamento parou de funcionar. Qual é a melhor abordagem?

A. **Capturar as classes de exceção tipadas do SDK, das mais específicas para as mais gerais, e decidir por tipo/status do erro.** [CORRETA]

B. Definir max_retries=0 no cliente, para que o SDK devolva o erro como JSON bruto em vez de lançar exceção.

C. Extrair o código HTTP com regex a partir do texto da mensagem de erro.

D. Adicionar mais substrings à checagem para cobrir as novas redações.

Resposta correta: A. Correta (A): o SDK levanta exceções tipadas por status/tipo de erro, e a documentação manda capturar essas classes (das mais específicas para as mais gerais) em vez de comparar o texto das mensagens. D e C continuam dependendo de texto, que pode mudar. B está errada: max_retries só configura os retries automáticos; o SDK continua levantando exceções tipadas.
Fonte: https://platform.claude.com/docs/en/api/errors

Questão 7 · D2 Applications & Integration · Claude Application Design · média

Você extrai dados de contratos com a API usando saída estruturada. Muitos contratos não informam a data de término nem o foro, e o modelo às vezes inventa valores plausíveis para esses campos. Quais mudanças de schema reduzem o problema? Selecione 2.

A. Colocar todos os campos em required, para que o modelo sempre devolva um valor para cada um.

B. Adicionar minLength: 1 a todas as strings para impedir valores vazios.

C. **Modelar o campo tipo_rescisao como enum que inclua um valor explícito como "nao_informado", em vez de string livre.** [CORRETA]

D. **Deixar data_termino e foro fora de required, tornando-os opcionais, para que a ausência da informação possa ser representada.** [CORRETA]

E. Confiar que a saída estruturada garante que os valores extraídos sejam verdadeiros em relação ao documento.

Resposta correta: C, D. Corretas (D e C): campos opcionais e um enum com valor explícito de "não informado" dão ao modelo uma saída legítima quando o contrato é omisso, em vez de forçá-lo a preencher. A está errada: exigir tudo em required pressiona o modelo a inventar. B está errada: minLength está na lista de restrições de string não suportadas em saída estruturada e, mesmo se fosse, empurraria o modelo a preencher algo. E está errada: a saída estruturada impõe conformidade com o schema (formato); não faz o conteúdo ser verdadeiro em relação ao documento. As próprias técnicas para reduzir alucinações dizem que reduzem o problema, mas não o eliminam.
Fonte: https://platform.claude.com/docs/en/build-with-claude/structured-outputs

Questão 8 · D8 Tools & MCPs · Agentic Customization · fácil

Você criou a skill /deploy, que executa o procedimento de deploy em produção. Quer que ela rode apenas quando você digitar /deploy, nunca por iniciativa de Claude por achar que a conversa pede um deploy. O que fazer?

A. **Definir disable-model-invocation: true no frontmatter da skill.** [CORRETA]

B. Listar apenas ferramentas seguras em allowed-tools na skill.

C. Definir context: fork no frontmatter, para rodar em um subagent isolado.

D. Mover o procedimento para o CLAUDE.md, que só é lido quando você pede.

Resposta correta: A. Correta (A): com essa opção Claude não carrega a skill automaticamente, e ela só roda quando invocada com /deploy (recomendado para skills com efeitos colaterais). D está errada: CLAUDE.md é carregado em toda sessão. B está errada: allowed-tools pré-aprova ferramentas, não impede a invocação automática. C só muda onde a skill executa.
Fonte: https://code.claude.com/docs/en/skills

Questão 9 · D1 Agents & Workflows · Agent Construction with Claude · média

Um agente do Agent SDK foi configurado com max_turns=5 para corrigir testes. Em produção, algumas execuções terminam sem nenhum texto final. Selecione 2 afirmações corretas sobre esse comportamento e seu tratamento.

A. O SDK dobra automaticamente max_turns e continua o loop quando o limite é atingido.

B. **O ResultMessage ainda traz session_id, permitindo retomar a sessão com um limite maior para continuar o trabalho.** [CORRETA]

C. **A execução termina com um ResultMessage de subtype error_max_turns, sem o campo result; o código deve checar o subtype antes de ler result.** [CORRETA]

D. Aumentar max_budget_usd também eleva o limite de turnos, então basta subir o orçamento para o loop rodar por mais tempo.

E. max_turns conta apenas turnos com chamadas de ferramenta; a resposta final só de texto nunca consome turno.

Resposta correta: B, C. Corretas (C e B): estourar o limite gera error_max_turns, sem result, e todo ResultMessage carrega session_id para retomar. D está errada: turnos e orçamento são limites independentes, cada um com seu subtype de erro. A está errada: o SDK não altera o limite sozinho. E está errada: a contagem inclui um turno extra para a resposta final só de texto, e sem ele a execução termina sem resposta final.
Fonte: https://code.claude.com/docs/en/agent-sdk/agent-loop

Questão 10 · D2 Applications & Integration · Understanding Requirements · fácil

Um banco vai lançar um chatbot de suporte baseado em Claude. A área de compliance exige que "toda resposta dada a um cliente seja auditável". Qual decisão de projeto atende melhor a esse requisito?

- **A.** **Persistir por resposta: id da conversa, versão do prompt, modelo, request-id da API e mensagens, com retenção definida** [CORRETA]
- **B.** Registrar apenas o texto final exibido ao cliente, para reduzir o volume armazenado e o risco de reter dados sensíveis do banco
- **C.** Ativar streaming, pois os eventos SSE recebidos pelo navegador do cliente já constituem um registro auditável da interação com o banco
- **D.** Confiar nos logs mantidos pela Anthropic e solicitá-los ao provedor sempre que ocorrer uma auditoria interna, externa ou regulatória

Resposta correta: A. Correta (A): auditabilidade é um requisito não funcional que vira dados rastreáveis sob seu controle: qual prompt/modelo gerou a resposta, o request-id (retornado em todo response) e o conteúdo, com política de retenção. D está errada porque a auditoria depende de registros próprios que você consiga produzir; o request-id serve para suporte, não substitui seu log. B está errada porque sem prompt, modelo e contexto não dá para explicar nem reproduzir por que aquela resposta foi dada. C está errada porque streaming é uma técnica de entrega e não gera registro durável nem confiável no lado do cliente.
Fonte: https://platform.claude.com/docs/en/api/errors

Questão 11 · D3 Claude Code · Claude Code · média

A equipe quer um procedimento /release que só rode quando alguém digitar o comando (nunca por decisão do Claude) e cujo material de referência longo (checklist e templates) não ocupe contexto em toda sessão. Selecione 2 práticas adequadas.

- **A.** Colar o procedimento completo e os templates no CLAUDE.md do projeto, para o Claude consultá-los.
- **B.** Definir user-invocable: false, para que a skill só seja usada de forma deliberada pela equipe.
- **C.** Usar allowed-tools no frontmatter para controlar quando o Claude pode acionar a skill.
- **D.** **Criar .claude/skills/release/SKILL.md com disable-model-invocation: true.** [CORRETA]
- **E.** **Manter checklist e templates em arquivos separados na pasta da skill, citados no SKILL.md.** [CORRETA]

Resposta correta: D, E. Corretas (D e E): disable-model-invocation: true deixa a skill apenas para invocação manual, e arquivos de apoio na pasta da skill só são lidos quando necessários, mantendo o SKILL.md enxuto. B faz o oposto: esconde a skill do menu / e a deixa só para o Claude. A carrega o conteúdo em toda sessão. C apenas pré-aprova ferramentas durante o turno; não controla quem aciona a skill.
Fonte: https://code.claude.com/docs/en/skills

Questão 12 · D5 Model Selection & Optimization · Cost and Token Management · difícil

Um job noturno envia 50 mil requisições ao Message Batches API, todas com o mesmo documento de 30k tokens como prefixo e perguntas diferentes. Não há urgência de resposta. Selecione 3 práticas corretas para minimizar o custo.

A. Dentro do lote, fazer uma requisição com max_tokens 0 para pré-aquecer o cache antes das demais requisições.

B. **Incluir blocos cache_control idênticos em todas as requisições do lote para aumentar a chance de hits.** [CORRETA]

C. Contar com hits garantidos, já que o lote processa as requisições em ordem sequencial rigorosa e previsível.

D. **Considerar o TTL de 1 hora, pois lotes podem levar mais de 5 minutos para processar.** [CORRETA]

E. **Usar o Batches API, que dá 50% de desconto para trabalho assíncrono sem urgência.** [CORRETA]

Resposta correta: B, D, E. Corretas (E, B e D): batch dá desconto de 50%; cache_control idêntico e TTL de 1 hora aumentam as chances de hit em lotes. C está errada porque requisições do lote podem rodar em paralelo e em qualquer ordem, então hits são best-effort. A está errada porque max_tokens 0 não é suportado dentro de um lote (a entrada expiraria antes do uso).

Fonte: https://platform.claude.com/docs/en/build-with-claude/batch-processing

Questão 13 · D2 Applications & Integration · Claude Application Design · média

Seu time habilitou 12 plugins no Claude Code e nota que as sessões começam com menos espaço de contexto, mesmo em tarefas que usam poucos deles. Quais afirmações sobre o custo dos plugins são verdadeiras? Selecione 2.

A. **Dá para parar o custo sem desinstalar, desabilitando o plugin com /plugin ou com claude plugin disable.** [CORRETA]

B. **O nome e a descrição de cada skill que Claude pode invocar sozinho ficam no contexto a cada turno, mesmo em sessões em que nada do plugin é usado.** [CORRETA]

C. Desinstalar é a única forma de remover o custo de contexto de um plugin.

D. Um plugin só afeta a sessão quando uma de suas skills é invocada; hooks e servidores MCP ficam inativos até lá.

E. O texto completo de todas as skills dos plugins habilitados é carregado no início de cada sessão.

Resposta correta: A, B. Corretas (B e A): descrições de skills invocáveis pelo modelo ocupam contexto o tempo todo, e desabilitar o plugin interrompe o custo sem desinstalar. E está errada: o texto completo só carrega quando a skill é usada. D está errada: um plugin habilitado faz parte de toda sessão, com hooks disparando e servidores MCP rodando. C contradiz A: desabilitar basta.

Fonte: https://code.claude.com/docs/en/plugins/overview

Questão 14 · D6 Prompt & Context Engineering · Context Engineering · difícil

Um dev está estimando quanto da janela de contexto uma requisição de agente consome. A requisição tem system prompt, 12 definições de tools, histórico com blocos tool_result e o texto que Claude vai gerar no turno. Quais afirmações sobre o que conta para a janela de contexto são corretas? Selecione 3.

A. **O conteúdo dos blocos tool_result no histórico conta como qualquer outra mensagem.** [CORRETA]

B. **O texto que Claude gera no turno, incluindo o thinking, conta para a janela.** [CORRETA]

C. **As definições de tools contam para a janela, assim como o system prompt.** [CORRETA]

D. Tokens lidos de um prefixo em cache (prompt caching) deixam de ocupar a janela; o cache altera apenas o preço.

E. Só as mensagens user/assistant contam; system prompt e definições de tools ficam fora da conta.

Resposta correta: A, B, C. Corretas (B, C, A): tudo na requisição conta para a janela: system prompt, cada mensagem em messages (incluindo tool results), as definições de tools, e também a saída gerada no turno, thinking incluído. D está errada: prefixos em cache continuam ocupando a janela; o caching muda o que você paga por esses tokens, não se eles contam. E está errada: system prompt e definições de tools contam, o que torna as definições de tools um custo de contexto real em agentes com muitas tools.

Fonte: https://platform.claude.com/docs/en/build-with-claude/context-windows

Questão 15 · D7 Security & Safety · Identity, Secrets and Key Management · fácil

Um time quer reduzir o risco de vazar chaves de API da Claude antes que aconteça qualquer incidente. Quais duas práticas de prevenção são recomendadas? Selecione 2.

A. Manter a mesma chave sem rotação até suspeitar de vazamento.

B. **Verificar regularmente os repositórios com uma ferramenta como Gitleaks.** [CORRETA]

C. Codificar a chave em base64 no bundle do front-end para que não fique legível.

D. Compartilhar uma única chave do time por chat para acelerar o onboarding.

E. **Carregar a chave por variável de ambiente e listar .env no .gitignore.** [CORRETA]

Resposta correta: B, E. Corretas (E, B): variáveis de ambiente com .env no .gitignore evitam que a chave entre no repositório, e a verificação periódica de repositórios pega segredos commitados por engano. C está errada: base64 é codificação, não proteção, e chave em código de cliente fica exposta. D está errada: chaves não devem ser compartilhadas, assim como senhas pessoais. A está errada: a recomendação é rotacionar em um cronograma regular, não apenas reagir a suspeitas.

Fonte: https://support.claude.com/en/articles/9767949-api-key-best-practices-keeping-your-keys-safe-and-secure

Questão 16 · D6 Prompt & Context Engineering · Prompt Engineering · média

Um prompt cola três contratos como texto corrido separados por linhas '---', seguidos das instruções. Claude às vezes atribui uma cláusula ao contrato errado e confunde trechos dos contratos com as instruções. Qual a melhor correção?

A. Mover os três contratos para o system prompt, para que tenham mais autoridade que as instruções em toda requisição.

B. **Envolver cada contrato em <document index="n"> com <source> e <document_content>, dentro de <documents>.** [CORRETA]

C. Repetir as instruções depois de cada um dos contratos para que Claude não as confunda com o texto dos contratos.

D. Trocar os separadores '---' por banners em letras maiúsculas como '=== CONTRATO 2 ==='.

Resposta correta: B. Correta (B): tags XML descritivas e consistentes separam instruções, contexto e dados sem ambiguidade; para vários documentos, o padrão recomendado é <document> com <source> e <document_content> dentro de <documents>. D continua sendo texto livre sem delimitação inequívoca, e não identifica fonte de forma estruturada. A não resolve a confusão entre documentos e ainda mistura dados com instruções. C infla o prompt e não dá fronteiras claras entre os documentos.

Fonte: https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices

Questão 17 · D7 Security & Safety · AI Application Security · fácil

Um agente de suporte lê tickets de clientes e tem a tool query_db com credencial de administrador do banco e a tool send_email sem restrição de destinatário. A equipe teme que um ticket malicioso faça o agente exfiltrar dados. Qual a melhor abordagem?

A. Pedir educadamente nos tickets de teste que o agente ignore instruções contidas no texto do cliente.

B. Adicionar ao system prompt a frase 'nunca envie dados a terceiros' e manter as permissões atuais.

C. **Credencial somente leitura nas tabelas necessárias, tools estreitas e nenhum segredo desnecessário.** [CORRETA]

D. Registrar logs de auditoria de todas as chamadas de tool e revisá-los semanalmente, mantendo as permissões.

Resposta correta: C. Correta (C): o princípio de menor privilégio faz com que uma injeção bem-sucedida cause dano mínimo: não dar acesso a segredos desnecessários e restringir permissões ao mínimo. B depende só da obediência do modelo, sem controle técnico. D é monitoramento: camada complementar útil, mas que descobre o vazamento depois que ele ocorreu e mantém as permissões excessivas. A não é controle de segurança; instruções em linguagem natural não limitam o que as credenciais permitem.
Fonte: https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/mitigate-jailbreaks

Questão 18 · D8 Tools & MCPs · Agentic Customization · média

Sua equipe tem a função lookup_customer, que consulta o CRM. Hoje só um serviço interno, que chama a Messages API, a usa. Em breve ela também será usada no Claude Code, no Claude Desktop e em outro app. Quais motivos justificam empacotá-la como servidor MCP? Selecione 2.

A. **Vários hosts (Claude Code, Claude Desktop, outros apps) usam o mesmo servidor, sem reimplementar a tool.** [CORRETA]

B. Tools definidas no código da aplicação não conseguem acessar APIs internas, como o CRM da empresa; só servidores MCP conseguem.

C. A Messages API só aceita tools que venham de servidores MCP, não funções definidas pelo seu código.

D. **A conexão e a autenticação com o CRM ficam em um só lugar, no servidor, e não duplicadas em cada cliente.** [CORRETA]

E. Uma tool exposta por MCP é chamada pelo modelo com mais precisão do que uma tool definida no código da própria aplicação.

Resposta correta: A, D. Corretas (A e D): na arquitetura MCP, um host (Claude Code, Claude Desktop) cria um cliente por servidor, e um servidor remoto atende vários clientes; a autenticação é tratada na camada de transporte do servidor, e não duplicada em cada cliente (a visão geral de extensões do Claude Code também descreve MCP como conexão e autenticação a cargo do servidor). E está errada: o protocolo não torna a chamada mais precisa; isso depende de nome, descrição e schema da tool. B está errada: uma tool no seu código pode chamar qualquer API que seu código alcance. C está errada: tools declaradas em tools na Messages API, com execução no seu código, funcionam sem MCP; para um único app, MCP seria exagero.
Fonte: https://modelcontextprotocol.io/docs/learn/architecture

Questão 19 · D6 Prompt & Context Engineering · Output Handling · média

Um assistente responde perguntas sobre um relatório anual e, às vezes, afirma com tom confiante números que não estão no relatório. Quais duas medidas ajudam a reduzir e detectar isso? Selecione 2.

A. **Rodar o prompt várias vezes e revisar as respostas divergentes, que podem indicar alucinação.** [CORRETA]

B. Instruir Claude a completar com conhecimento geral quando o relatório não trouxer o dado, para manter a utilidade.

C. Dispensar a validação humana depois de restringir as respostas ao relatório, pois o risco deixa de existir.

D. Pedir que Claude informe uma confiança de 0 a 100 e aceitar automaticamente respostas acima de 90, sem revisão.

E. **Permitir que Claude responda que não há informação suficiente quando o relatório não trouxer o dado.** [CORRETA]

Resposta correta: A, E. Corretas (E, A): permitir 'não sei' reduz drasticamente informação inventada, e comparar várias execuções (best-of-N) expõe inconsistências que indicam alucinação. D está errada: a página não propõe confiança autodeclarada como verificação; as técnicas listadas ancoram a resposta no texto-fonte (citações, citações diretas) ou comparam execuções, e nada disso é substituído por um número informado pelo próprio modelo. B vai no sentido oposto: a recomendação é restringir ao conteúdo fornecido. C está errada: as técnicas reduzem, mas não eliminam alucinações; informações críticas ainda exigem validação.
Fonte: https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/reduce-hallucinations

Questão 20 · D2 Applications & Integration · Understanding Requirements · média

O requisito de um chatbot de suporte diz apenas: "deve ser rápido e estar sempre disponível". Antes de projetar, a equipe precisa transformá-lo em requisitos verificáveis. Selecione 2 abordagens adequadas.

A. Fixar 100% de disponibilidade como meta, sem nenhuma resposta de erro tolerada em qualquer período, e provisionar capacidade para isso

B. **Definir p95 de tempo até o primeiro token abaixo de um valor acordado, sob carga especificada, medido em produção** [CORRETA]

C. Exigir latência média abaixo do alvo, já que a média resume bem a experiência de todos os usuários e é simples de acompanhar

D. **Definir um SLO de disponibilidade (% de requisições com sucesso no mês) e o comportamento degradado em 429 ou 529** [CORRETA]

E. Escolher sempre o maior modelo disponível, pois a capacidade extra garante respostas rápidas e disponibilidade em horários de pico

Resposta correta: B, D. Corretas (B e D): percentis com condição de carga e ponto de medição tornam a latência testável, e um SLO com comportamento degradado explícito (fila, mensagem, handoff humano) torna a disponibilidade realista. C está errada porque a média esconde a cauda longa de respostas lentas. A está errada porque 100% é irreal e encarece a solução; o recomendado é um SLO com error budget. E está errada porque escolher o maior modelo não garante latência baixa nem disponibilidade; metas verificáveis é que tornam isso testável.
Fonte: https://sre.google/sre-book/service-level-objectives/

Questão 21 · D2 Applications & Integration · Configuration Management · difícil

O repositório tem .claude/settings.json versionado com permissions.allow ["Bash(npm run lint)"] e permissions.deny ["Read(./.env)"]. Uma pessoa nova clona o repo e abre o Claude Code, ainda sem ter confiado (trust) na pasta. O que acontece?

A. Nenhuma das duas vale até a confiança, então o .env pode ser lido nesse intervalo.

B. A regra allow vale de imediato, enquanto a deny só vale após confiar na pasta, por exigir confirmação.

C. **A regra deny vale de imediato; a regra allow só passa a valer depois de confiar na pasta, então o lint ainda pede confirmação.** [CORRETA]

D. As duas regras valem imediatamente, pois o arquivo está versionado no repositório.

Resposta correta: C. Correta (C): regras allow do arquivo compartilhado aguardam o trust da pasta, enquanto deny e ask se aplicam logo, o que mantém a restrição segura por padrão. D ignora a etapa de trust para allow. A deixaria uma brecha de segurança que o design evita. B inverte a lógica: o que é permissivo espera confirmação, não o restritivo.
Fonte: https://code.claude.com/docs/en/settings

Questão 22 · D8 Tools & MCPs · MCP Server Development · difícil

Sua empresa quer um servidor MCP somente-leitura sobre o ERP, usado por centenas de devs no Claude Code (e por outros apps), com identificação do usuário em cada acesso. Qual desenho é o mais adequado?

A. Servidor remoto com transporte SSE, por ser o mais recomendado para serviços remotos que mantêm conexões longas.

B. Um único processo stdio em um host central, ao qual todos os devs se conectam ao mesmo tempo, com a credencial do ERP configurada nele.

C. **Servidor remoto Streamable HTTP com autenticação HTTP padrão (OAuth) e clientes conectando por URL.** [CORRETA]

D. Servidor stdio que cada dev executa localmente, com a credencial do ERP em variável de ambiente da própria máquina.

Resposta correta: C. Correta (C): a arquitetura MCP descreve servidores remotos com transporte Streamable HTTP, que tipicamente atendem muitos clientes e suportam autenticação HTTP padrão (bearer tokens, API keys, headers customizados), com OAuth recomendado para obter tokens. D espalha a credencial do ERP em cada máquina e não atende outros apps por URL. A está errada: nos docs de MCP do Claude Code o SSE está obsoleto e HTTP é o recomendado para servidores remotos. B está errada: servidores locais com transporte stdio tipicamente atendem um único cliente MCP.
Fonte: https://modelcontextprotocol.io/docs/learn/architecture

Questão 23 · D5 Model Selection & Optimization · Cost and Token Management · média

Uma ferramenta interna chama o Claude uma vez a cada ~30 minutos com o mesmo prefixo grande e cache_control com o TTL padrão. A cobrança subiu após habilitar caching e a resposta mostra cache_creation_input_tokens alto e cache_read_input_tokens sempre 0. Qual é a melhor interpretação e ação?

A. O prefixo é pequeno demais, por isso nada é lido; aumentar max_tokens resolve.

B. **O TTL padrão (5 minutos) expira antes da próxima chamada (~30 minutos): toda requisição paga a escrita no cache e nunca lê; a ação é remover o caching ou avaliar o TTL de 1 hora, que cobre esse intervalo entre chamadas.** [CORRETA]

C. Os campos de usage estão errados em modelos recentes; o cache está funcionando e a cobrança vai estabilizar.

D. É preciso adicionar mais breakpoints (até 4) para que o cache comece a ser lido.

Resposta correta: B. Correta (B): o TTL padrão é de 5 minutos e escritas custam mais que input normal; sem reuso dentro do TTL só há escrita, sem leituras baratas. O TTL de 1 hora é a opção documentada para intervalos maiores. C está errada porque os campos refletem o que ocorreu. D está errada porque mais breakpoints não evitam expiração. A está errada porque prefixo curto demais nem seria cacheado (cache_creation_input_tokens seria 0, ao contrário do observado) e max_tokens não influencia o cache.
Fonte: https://platform.claude.com/docs/en/build-with-claude/prompt-caching

Questão 24 · D2 Applications & Integration · Systems Life Cycle · difícil

Uma equipe guarda o system prompt de um extrator em um repositório e o edita no lugar sob o rótulo "v2". Uma reformulação "pequena" mudou o formato da saída, quebrou o parser downstream e foi para 100% do tráfego de uma vez, mesmo com os evals offline passando. A equipe decide adotar versionamento semântico (SemVer) para os prompts. Qual abordagem de versionamento e rollout é a melhor?

A. Escolher o número da versão pelo tamanho do diff: poucas linhas alteradas significam PATCH, mesmo que o formato da saída mude, e reservar MAJOR para reescritas extensas do prompt

B. Liberar direto para 100% quando os evals offline passarem, pois eles substituem a observação em produção

C. Manter o rótulo "v2" e sobrescrevê-lo a cada ajuste, pois o histórico do Git já permite voltar a qualquer versão anterior

D. **Declarar o formato de saída como API pública do prompt, versões publicadas imutáveis (formato quebrado = MAJOR), evals como portão e canário com rollback** [CORRETA]

Resposta correta: D. Correta (D): o SemVer exige declarar uma API pública, proíbe alterar o conteúdo de uma versão já publicada e reserva MAJOR para mudanças incompatíveis; aplicado a prompts, o contrato que o parser consome é o formato de saída. Evals são o portão e o canário com monitoramento e rollback (prática de rollout gradual, que não faz parte do SemVer) limita o dano que escapar. C está errada porque sobrescrever o rótulo viola a imutabilidade e deixa de dizer qual prompt gerou cada saída em produção. A está errada porque o SemVer classifica pela compatibilidade da API e não pelo tamanho do diff: uma linha pode quebrar o contrato. B está errada porque evals offline não cobrem a distribuição real de entradas; o canário detecta o que escapou.
Fonte: https://semver.org/

Questão 25 · D1 Agents & Workflows · Agent Patterns & Frameworks · média

Um subagente de revisão foi definido em .claude/agents/reviewer.md, com description "Revisor somente leitura; nunca altera arquivos" e sem campo tools. Mesmo assim ele usou Write e Edit. A equipe exige que essas duas ferramentas fiquem indisponíveis para ele. Selecione 2 ajustes eficazes.

A. **Adicionar disallowedTools: Write, Edit ao frontmatter do subagente.** [CORRETA]

B. Definir isolation: worktree, para que as edições nunca cheguem ao checkout principal.

C. Reforçar no campo description que o subagente jamais deve usar Write ou Edit.

D. **Adicionar tools: Read, Grep, Glob ao frontmatter do subagente.** [CORRETA]

E. Definir model: haiku, pois um modelo menor tende a editar menos.

Resposta correta: A, D. Corretas (D e A): sem tools, o subagente herda todas as ferramentas disponíveis; uma allowlist (tools) ou uma denylist (disallowedTools) remove Write e Edit de fato. C está errada: description orienta quando o Claude delega ao subagente, não restringe ferramentas. B isola o diretório, mas Write e Edit continuam disponíveis. E muda o modelo, sem relação com permissões de ferramenta.
Fonte: https://code.claude.com/docs/en/sub-agents

Questão 26 · D5 Model Selection & Optimization · LLM Fundamentals · difícil

Um app de FAQ em um modelo tier Sonnet usa adaptive thinking. Monitorando, você vê blocos de thinking em muitas perguntas triviais, aumentando latência e custo de output. Adicionar ao system prompt "não pense, responda direto" não reduziu de forma confiável. Qual é a melhor abordagem?

A. Definir um budget_tokens pequeno em thinking, para limitar o raciocínio a poucos tokens.

B. **Reduzir o nível de effort (output_config.effort), medir o impacto na qualidade e só então complementar com orientação no prompt, se necessário.** [CORRETA]

C. Reforçar o system prompt com mais ênfase ("NUNCA pense") e assumir que o modelo obedecerá sempre.

D. Aumentar max_tokens para que o thinking termine mais rápido e libere a resposta.

Resposta correta: B. Correta (B): effort é a alavanca calibrada para a frequência e a profundidade do thinking; a orientação por prompt é secundária, sensível à redação e deve ser medida antes de ir para produção. C está errada: segundo a documentação, em modelos como os Sonnet atuais, pedir no prompt para pensar menos não reduz o thinking de forma confiável. A está errada: a documentação diz que não se define orçamento de thinking nesses modelos (thinking enabled com budget_tokens é rejeitado com 400); o custo é limitado por max_tokens e effort. D está errada: max_tokens é só um teto de saída (thinking incluído), não faz o modelo pensar menos.
Fonte: https://platform.claude.com/docs/en/build-with-claude/thinking-steering-and-cost

Questão 27 · D7 Security & Safety · Claude Hooks · média

Um time cria um hook PreToolUse que detecta quando Claude Code tenta ler um arquivo .env, imprime uma mensagem no stderr e termina com exit 1. Mesmo assim, o arquivo continua sendo lido. Qual a melhor correção?

A. Manter exit 1, que por ser o código convencional de falha em Unix já bloqueia a ação.

B. Mover a lógica para um hook PostToolUse, que roda depois que a tool executou e recebe seu resultado.

C. **Usar exit 2: em PreToolUse bloqueia a chamada e o stderr vira a mensagem.** [CORRETA]

D. Imprimir a mensagem no stdout e terminar com exit 0, pois a mensagem sozinha cancela a tool call.

Resposta correta: C. Correta (C): exit 2 é o mecanismo de bloqueio e, em PreToolUse, bloqueia a chamada e o stderr vira o motivo entregue a Claude. A está errada: qualquer código diferente de 0 e 2 é um erro não bloqueante e a tool call prossegue, apesar de 1 ser o código de falha convencional. B está errada: PostToolUse roda depois da execução e não pode bloquear (a ferramenta já rodou). D está errada: exit 0 significa sucesso e a ação segue, a menos que o JSON de saída a negue; texto no stdout não cancela nada.
Fonte: https://code.claude.com/docs/en/hooks

Questão 28 · D8 Tools & MCPs · Tool Implementation · difícil

A tool get_invoice(invoice_id) roda no seu backend, com strict: true e invoice_id como string no schema. Outra tool devolve, em um tool_result, o texto de um e-mail de terceiros que tenta induzir Claude a pedir a fatura de OUTRO cliente. Quais afirmações são verdadeiras? Selecione 2.

A. strict: true garante que o invoice_id enviado pertença ao cliente autenticado, validando a propriedade da fatura.

B. **strict: true garante que as entradas da tool correspondam ao schema declarado.** [CORRETA]

C. O conteúdo não confiável deve ser colocado no system prompt, que é o lugar mais seguro para ele, com prioridade maior.

D. **Conteúdo externo em tool_result, como e-mails, deve ser tratado como não confiável, pois pode embutir instruções (injeção indireta de prompt).** [CORRETA]

E. O e-mail está dentro de um tool_result, então o seu conteúdo é confiável por vir do seu próprio backend, que o entregou.

Resposta correta: B, D. Corretas (B e D): strict garante conformidade com o schema, e conteúdo externo em tool_result deve ser tratado como não confiável (injeção indireta). A está errada: o schema só descreve forma (aqui, uma string); quem decide se a fatura pertence ao cliente não é o strict. E está errada: estar num tool_result não torna o conteúdo confiável; a origem é externa. C está errada: a orientação é manter conteúdo não confiável dentro de blocos tool_result, e não em system prompts.
Fonte: https://platform.claude.com/docs/en/agents-and-tools/tool-use/handle-tool-calls

Questão 29 · D4 Eval, Testing & Debugging · Debugging & Error Handling · difícil

Trace de um agente: nos turnos 1, 2 e 3 Claude chama search_docs com {"query":"rate limits"} e recebe is_error=true: "Missing required 'repo' parameter". No turno 4 ele pede desculpas e desiste. O schema de search_docs exige repo, mas a descrição é "Search docs." e repo não tem description. Qual a melhor correção?

A. Aumentar max_turns para que Claude acabe acertando o parâmetro em mais tentativas.

B. Aumentar tentativas e backoff no executor da ferramenta, pois a integração está falhando.

C. Trocar para tool_choice forçado em search_docs, pois Claude escolheu a ferramenta errada.

D. **Detalhar a descrição da ferramenta e do parâmetro repo (o que é, valores válidos) e habilitar strict: true, tratando o problema como definição insuficiente.** [CORRETA]

Resposta correta: D. Correta (D): a ferramenta certa foi escolhida; o que falta é informação para preencher repo. A documentação recomenda descrições mais detalhadas, e strict: true garante entradas conforme o schema, evitando parâmetros ausentes. C está errada: não houve escolha errada de ferramenta, e o forçado pode ser rejeitado em modelos novos. B está errada: o executor nem chegou a rodar; o erro é de entrada. A está errada: Claude já repetiu e desistiu, e mais turnos repetem o mesmo erro.
Fonte: https://platform.claude.com/docs/en/agents-and-tools/tool-use/handle-tool-calls

Questão 30 · D8 Tools & MCPs · Tool Implementation · média

Seu app usa a Messages API com uma tool client issue_refund, que movimenta dinheiro. Você quer que um humano aprove cada reembolso antes de ele acontecer. Qual abordagem?

A. Transformar issue_refund em server tool, para que a execução e a aprovação fiquem a cargo da Anthropic, sem código próprio para essa etapa.

B. **No código que trata tool_use, pausar antes de executar, pedir aprovação e, se negada, devolver tool_result com is_error true.** [CORRETA]

C. Deixar o handler executar o reembolso e, se o humano discordar depois, pedir a Claude que estorne.

D. Escrever na description da tool "sempre peça aprovação ao usuário antes de chamar" e confiar que Claude cumprirá em cada conversa com clientes.

Resposta correta: B. Correta (B): com client tools, quem executa a tool é o seu código, então é lá que fica o ponto de controle; a negativa volta como tool_result com is_error para Claude se adaptar. D é só um pedido no prompt, sem garantia. C faz a ação irreversível antes da decisão. A está errada: server tools são executadas internamente por Claude/API, sem passar pelo seu código, então não há onde inserir o seu fluxo de aprovação.
Fonte: https://platform.claude.com/docs/en/agents-and-tools/tool-use/handle-tool-calls

Questão 31 · D2 Applications & Integration · Claude API Mechanics · média

Um app faz streaming de uma resposta em que Claude chama uma ferramenta. O código executa json.loads em cada partial_json recebido nos eventos input_json_delta e lança exceção em quase todo evento. Qual a abordagem correta?

A. Capturar a exceção e usar o último fragmento que foi parseado com sucesso como os argumentos da ferramenta

B. **Acumular os fragmentos por índice de bloco, fazer o parse ao receber content_block_stop e só então executar a ferramenta** [CORRETA]

C. Executar a ferramenta assim que o primeiro campo aparecer no fragmento, com um regex, para reduzir a latência

D. Desativar o streaming apenas durante o tool use, pois não há forma de obter o JSON dos argumentos em requisições com stream

Resposta correta: B. Correta (B): os deltas são fragmentos de string que só formam um JSON válido ao final do bloco; acumule-os por índice e faça o parse em content_block_stop (ou use os helpers do SDK). C está errada porque executar com argumentos parciais pode rodar ação incorreta ou incompleta. A está errada porque o último parse bem-sucedido é um objeto truncado e não os argumentos reais. D está errada porque o streaming de tool use é suportado e o JSON completo é obtido ao acumular os fragmentos.
Fonte: https://platform.claude.com/docs/en/build-with-claude/streaming

Questão 32 · D1 Agents & Workflows · Agent Architecture · fácil

Uma equipe usa uma única chamada ao Claude para revisar mudanças de código em busca de vulnerabilidades. O problema é a variância: em execuções repetidas, achados reais ora aparecem, ora somem. Que padrão de workflow ataca esse problema?

A. **Voting: rodar a mesma revisão várias vezes de forma independente e sinalizar a mudança quando um número suficiente de execuções apontar problema.** [CORRETA]

B. Routing: classificar cada mudança por linguagem e encaminhá-la a um prompt de revisão especializado.

C. Prompt chaining: encadear a revisão por um segundo prompt que reescreve e refina o texto do relatório.

D. Sectioning: dividir o diff em partes independentes e processar cada parte em paralelo, cada uma com um prompt diferente.

Resposta correta: A. Correta (A): voting é a variação de parallelization que executa a mesma tarefa várias vezes para obter saídas diversas e agregá-las, o que reduz a variância de uma chamada única. D (sectioning) paraleliza subtarefas distintas para ganhar velocidade ou foco, mas cada parte continua sendo avaliada uma única vez. C melhora a redação do relatório, não a confiabilidade da detecção. B especializa prompts por tipo de entrada, mas cada entrada ainda é julgada uma só vez.
Fonte: https://www.anthropic.com/engineering/building-effective-agents

Questão 33 · D5 Model Selection & Optimization · LLM Fundamentals · média

Uma aplicação usa um modelo com janela de 1M tokens. Cada requisição envia um prefixo de 300k tokens marcado com prompt caching, mais a pergunta do usuário, com adaptive thinking ativo e max_tokens definido. Um colega faz afirmações sobre o uso da janela de contexto. Selecione 2 afirmações corretas.

A. **Os tokens de thinking gerados contam para a janela de contexto e fazem parte do limite de max_tokens.** [CORRETA]

B. A janela limita apenas o input; os tokens de saída são contabilizados em um limite separado de geração.

C. **Os 300k tokens do prefixo lidos do cache continuam ocupando a janela; o caching muda o preço, não o espaço.** [CORRETA]

D. Tokens lidos do cache não contam para a janela, pois não são reprocessados como input normal pelo modelo.

E. Se o input sozinho exceder a janela, a API trunca silenciosamente as mensagens mais antigas e responde normalmente.

Resposta correta: A, C. Corretas (C e A): a janela inclui system prompt, mensagens, tools e a saída gerada (inclusive thinking); tokens em cache seguem ocupando espaço, e só o preço muda. D está errada porque cache não reduz uso da janela. E está errada porque input maior que a janela retorna erro 400 (prompt is too long), sem truncar. B está errada porque a janela cobre input e output juntos.
Fonte: https://platform.claude.com/docs/en/build-with-claude/context-windows

Questão 34 · D5 Model Selection & Optimization · Technical Fundamentals · média

Antes de enviar prompts grandes, sua API interna chama o endpoint de contagem de tokens para decidir roteamento e checar limites. Selecione 2 afirmações corretas sobre esse endpoint.

A. **Retorna uma estimativa; o número real de tokens de input na criação pode diferir um pouco.** [CORRETA]

B. Retorna o número exato que será faturado, sempre idêntico ao usage da resposta real do modelo.

C. **É gratuito, com limite de requisições por minuto próprio, separado do da criação de mensagens.** [CORRETA]

D. Respeita blocos cache_control, então o resultado já reflete o desconto de leitura de cache na fatura.

E. Aceita a mesma requisição da Messages API, inclusive com server tools como web search e code execution.

Resposta correta: A, C. Corretas (C e A): a contagem é gratuita com rate limit separado e é uma estimativa. B está errada porque não é garantia de igualdade com o faturado. D está errada porque o caching só ocorre na criação real da mensagem. E está errada porque server tools (web search, code execution etc.) geram invalid_request_error nesse endpoint.
Fonte: https://platform.claude.com/docs/en/build-with-claude/token-counting

Questão 35 · D1 Agents & Workflows · Agent Patterns & Frameworks · fácil

Um agente do Agent SDK recebeu no prompt inicial a regra "escreva sempre as notas de release em português". Em sessões longas, após eventos compact_boundary, ele passa a escrever em inglês. Qual a melhor correção?

A. Definir effort como max para que o modelo lembre melhor das instruções iniciais ao longo da sessão.

B. **Colocar a regra em CLAUDE.md carregado via setting_sources, reinjetado em cada requisição.** [CORRETA]

C. Aumentar max_budget_usd para que o contexto original não seja resumido pela compactação automática.

D. Retomar a sessão a cada turno com o session_id para restaurar o prompt original antes de responder.

Resposta correta: B. Correta (B): a compactação substitui mensagens antigas por um resumo, e instruções do início da conversa podem não sobreviver; regras persistentes devem ficar no CLAUDE.md, reinjetado a cada requisição. C não controla compactação: orçamento limita gasto. A muda a profundidade de raciocínio, não o que permanece no contexto. D restaura o contexto já compactado, com o resumo no lugar das mensagens originais.
Fonte: https://code.claude.com/docs/en/agent-sdk/agent-loop

Questão 36 · D5 Model Selection & Optimization · Technical Fundamentals · média

Em um loop de agente, uma resposta chega com stop_reason "max_tokens" e o último bloco de conteúdo é um tool_use cujo input JSON parece cortado no meio. O que fazer?

A. **Tratar a resposta como incompleta e repetir a requisição com um max_tokens maior, em vez de executar a ferramenta.** [CORRETA]

B. Enviar o tool_use parcial de volta como tool_result e pedir ao modelo que complete o JSON.

C. Executar a ferramenta com o input parcial, assumindo que os campos que chegaram bastam.

D. Ignorar o stop_reason, pois o SDK completa automaticamente chamadas de ferramenta truncadas.

Resposta correta: A. Correta (A): max_tokens significa que a resposta atingiu o teto; com tool_use truncado, a recomendação é reenviar com limite maior. C está errada porque o input incompleto pode executar ações erradas. B está errada porque tool_result é a saída de uma ferramenta já chamada, não um canal para completar JSON. D está errada porque a documentação orienta reenviar com max_tokens maior e não prevê que o SDK complete um tool_use truncado.
Fonte: https://platform.claude.com/docs/en/build-with-claude/handling-stop-reasons

Questão 37 · D2 Applications & Integration · Software Engineering Foundations · média

Um serviço Node processa 2.000 documentos disparando todas as chamadas à API com Promise.all e recebe uma enxurrada de erros 429. Selecione 3 correções adequadas.

A. **Aumentar a carga de forma gradual em vez de saltar do zero ao pico, evitando também os limites de aceleração** [CORRETA]

B. Repetir a requisição imediatamente em loop, até uma das tentativas passar

C. Adicionar retries independentes em cada camada (fila, serviço e cliente), para que cada falha tenha mais chances de passar

D. **Limitar a concorrência com um pool de workers ou semáforo, dimensionado abaixo dos limites de taxa da organização** [CORRETA]

E. **Aplicar retry com backoff exponencial e jitter, respeitando o header retry-after quando presente e com número máximo de tentativas** [CORRETA]

Resposta correta: A, D, E. Corretas (D, E e A): concorrência limitada evita estourar a taxa; backoff exponencial respeitando retry-after espaça as novas tentativas (o jitter é prática geral para não sincronizá-las); e rampa gradual evita os acceleration limits, conforme a página de rate limits. B está errada porque retentar sem espera piora a sobrecarga e continua recebendo 429. C está errada porque o SDK oficial já repete automaticamente falhas transitórias (2 vezes por padrão, respeitando retry-after); empilhar retries independentes em fila, serviço e cliente multiplica as tentativas por falha e agrava a sobrecarga.
Fonte: https://platform.claude.com/docs/en/api/rate-limits

Questão 38 · D1 Agents & Workflows · Agent Construction with Claude · fácil

Um agente do Agent SDK vai conduzir uma refatoração extensa. A equipe quer que ele leia o código e proponha um plano para aprovação humana antes de modificar qualquer arquivo. Qual permission_mode atende?

A. **plan: Claude explora e planeja, e as edições vão para o callback canUseTool.** [CORRETA]

B. dontAsk, que nunca pergunta e por isso nada não revisado chega a ser executado; o que não estiver pré-aprovado é negado.

C. bypassPermissions, que executa as chamadas sem pedir aprovação e é recomendado para testes locais.

D. acceptEdits, que aprova edições automaticamente para acelerar a iteração, mantendo o pedido de confirmação para outros comandos.

Resposta correta: A. Correta (A): o modo plan é o desenhado para produzir um plano primeiro, e as edições passam por canUseTool mesmo que uma regra de allow as cubra. D aprova edições sem revisão, o oposto do pedido. B é voltado a agentes headless sem ninguém para aprovar: nega o que pediria confirmação, em vez de apresentar um plano. C pula as verificações e só é indicado para ambientes isolados como CI ou contêineres.
Fonte: https://code.claude.com/docs/en/agent-sdk/agent-loop

Questão 39 · D2 Applications & Integration · Claude Application Design · fácil

No Claude Code do seu time há duas instruções: (1) "os testes rodam com pnpm test e os handlers da API ficam em src/api/handlers/"; (2) "respostas curtas, com pouca explicação", que alguns devs querem desligar quando estão aprendendo o código. Onde colocar cada uma?

A. (1) em uma skill com disable-model-invocation; (2) em um hook SessionStart.

B. **(1) em CLAUDE.md; (2) em um output style, que pode ser trocado quando se quiser.** [CORRETA]

C. (1) em um output style; (2) em CLAUDE.md.

D. As duas em CLAUDE.md, pois ele é carregado em toda sessão e cobre qualquer tipo de instrução.

Resposta correta: B. Correta (B): fatos e convenções do projeto (comandos de build, estrutura) são contexto permanente do CLAUDE.md; o formato e o tom das respostas são o papel de um output style, que é alternável. C inverte os papéis. D perde a possibilidade de ligar/desligar o estilo, já que CLAUDE.md é sempre carregado. A usa mecanismos para outro fim: uma skill sob demanda não garante que convenções estejam sempre no contexto, e hook é para automação em eventos, não para tom.
Fonte: https://code.claude.com/docs/en/features-overview

Questão 40 · D5 Model Selection & Optimization · Model Selection and Tradeoffs · fácil

Uma equipe vai migrar de um modelo de geração antiga para um tier de geração mais nova, mantendo os mesmos prompts. O orçamento de custo e a verificação de "cabe na janela" usam contagens de tokens medidas no modelo antigo. Qual é a melhor abordagem?

A. Reaproveitar as contagens, pois o número de tokens de um texto é o mesmo em qualquer modelo Claude, então não é preciso recontar nem refazer o custo.

B. Reavaliar só se o preço por milhão de tokens mudar; caso contrário, o orçamento continua válido.

C. **Recontar os prompts com o endpoint de contagem usando o modelo de destino, refazer estimativas de custo e janela e rodar evals.** [CORRETA]

D. Assumir que modelos mais novos sempre geram menos tokens para o mesmo texto e reduzir o orçamento.

Resposta correta: C. Correta (C): modelos mais novos podem usar tokenizer diferente, produzindo mais tokens para o mesmo texto; a doc manda recontar com o modelo-alvo, pois faturamento e janela refletem esse tokenizer. A está errada porque a contagem depende do modelo. B está errada porque o preço unitário igual ainda multiplica mais tokens. D está errada porque a direção pode ser a oposta (mais tokens).
Fonte: https://platform.claude.com/docs/en/build-with-claude/token-counting

Questão 41 · D2 Applications & Integration · Software Engineering Foundations · fácil

Um serviço usa output_config.format com um JSON schema e grava a saída do Claude direto no banco. Em produção, às vezes o parse falha em respostas cortadas ao meio. Qual a melhor abordagem?

A. Aplicar uma função que "conserta" JSON truncado, fechando chaves e aspas, e gravar o resultado reparado

B. Tratar a saída como sempre válida, já que structured outputs garantem o schema em qualquer resposta

C. Remover o schema e iniciar o turno do assistente com "{" para forçar o formato JSON nos modelos mais recentes

D. **Verificar o stop_reason antes de parsear, tratar max_tokens (retry com limite maior) e refusal como casos à parte, e validar na borda antes de gravar** [CORRETA]

Resposta correta: D. Correta (D): structured outputs garantem conformidade com o schema, exceto quando o stop_reason é max_tokens (saída incompleta) ou refusal (a recusa prevalece sobre o schema); por isso se checa o stop_reason e se valida na borda antes de persistir. B está errada porque esses dois casos quebram a garantia. C está errada porque o prefill de mensagem do assistente não é suportado nos modelos 4.6 em diante e retorna 400. A está errada porque reparar JSON truncado grava dados possivelmente incompletos como se fossem válidos.
Fonte: https://platform.claude.com/docs/en/build-with-claude/structured-outputs

Questão 42 · D2 Applications & Integration · Claude Application Design · fácil

Em uma sessão longa do Claude Code você pede um ajuste de formatação. Claude erra, você corrige, e ele erra a mesma coisa de novo. Qual a melhor ação a seguir?

A. **Rodar /clear e começar com um único prompt novo que já incorpora o que você aprendeu nas tentativas.** [CORRETA]

B. Fazer uma terceira correção, escrevendo a instrução em letras maiúsculas para dar ênfase.

C. Adicionar ao CLAUDE.md a frase em CAPS LOCK com IMPORTANT e continuar na mesma sessão.

D. Continuar corrigindo na mesma conversa, pois mais histórico de tentativas dá mais contexto ao modelo.

Resposta correta: A. Correta (A): depois de duas correções sem sucesso, a conversa já contém duas tentativas falhas que Claude continua relendo; limpar e reescrever o prompt funciona melhor. B e D só aumentam esse histórico. C está errada: modelos atuais seguem uma linha escrita de forma simples, e ênfase excessiva tende a fazer Claude aplicar a regra em excesso.
Fonte: https://code.claude.com/docs/en/best-practices

Questão 43 · D2 Applications & Integration · Claude API Mechanics · média

Um job com a Message Batches API de 5.000 requisições terminou com 4.700 resultados succeeded, 200 errored e 100 expired. Selecione 3 práticas corretas para consolidar e recuperar o trabalho.

A. **Reenviar em um novo batch os itens expired e com erro de servidor, e corrigir antes os itens com invalid_request_error** [CORRETA]

B. Associar cada resultado à entrada pela posição da linha no arquivo de resultados, já que ele segue a ordem de envio

C. **Associar cada resultado à entrada de origem pelo custom_id, pois a ordem dos resultados não é garantida** [CORRETA]

D. **Contar que itens errored e expired não são cobrados, então reprocessá-los não paga duas vezes pelo mesmo trabalho** [CORRETA]

E. Reenviar o batch inteiro de 5.000 itens para garantir que todos os resultados venham do mesmo processamento

Resposta correta: A, C, D. Corretas (C, A e D): o custom_id é a chave de correlação; itens expired ou com erro transitório podem ir a um novo batch, enquanto invalid_request exige corrigir a entrada; e requisições errored/expired não são cobradas. B está errada porque os resultados podem voltar em qualquer ordem. E está errada porque reprocessar os 4.700 itens bem-sucedidos desperdiça custo e tempo sem ganho.
Fonte: https://platform.claude.com/docs/en/build-with-claude/batch-processing

Questão 44 · D2 Applications & Integration · Claude API Mechanics · fácil

Em um chat com Claude, o dev chama a Messages API a cada turno enviando só a nova mensagem do usuário. O assistente "esquece" tudo o que foi dito antes. Qual a correção mais adequada?

A. Aumentar max_tokens, para que o modelo consiga reter mais contexto entre uma chamada e outra

B. Enviar a cada chamada apenas as falas anteriores do usuário, omitindo as respostas do assistente para economizar tokens

C. Reutilizar o mesmo header request-id em todos os turnos, para que a API associe as chamadas à mesma conversa

D. **Guardar a lista de mensagens da conversa e enviá-la completa a cada chamada, anexando a resposta do assistente e a nova fala do usuário** [CORRETA]

Resposta correta: D. Correta (D): a Messages API é stateless, então o histórico completo (turnos de user e assistant) deve ser enviado a cada requisição; quem guarda o estado é a sua aplicação. C está errada porque o request-id é um identificador de resposta usado em suporte e não vincula chamadas. A está errada porque max_tokens limita a saída, não cria memória entre requisições. B está errada porque, sem as respostas do assistente, o contexto fica incompleto e o modelo perde o que ele mesmo disse e concluiu.
Fonte: https://platform.claude.com/docs/en/build-with-claude/working-with-messages

Questão 45 · D6 Prompt & Context Engineering · Output Handling · fácil

Um serviço de triagem usa output_config.format com um JSON schema cujo campo "categoria" é um enum com os valores "Cobrança", "Suporte técnico" e "Outros". O código compara a categoria recebida por igualdade exata e, de vez em quando, cai no ramo "desconhecido" porque chegou, por exemplo, "Suporte Técnico". Qual a melhor correção?

A. **Comparar o valor sem diferenciar maiúsculas de minúsculas e evitar enum que difira só na capitalização.** [CORRETA]

B. Abandonar structured outputs e pedir JSON em texto livre, perdendo a conformidade de estrutura garantida pelo schema.

C. Assumir que o schema garante a grafia exata do enum e tratar a divergência como bug da API a ser reportado ao suporte.

D. Tratar como erro transitório e repetir a requisição até o valor vir exatamente como no schema, com limite de tentativas.

Resposta correta: A. Correta (A): a documentação de structured outputs avisa que a capitalização de valores string de enum e const não é garantida, e recomenda comparar sem diferenciar maiúsculas/minúsculas e evitar valores que difiram só por capitalização. D está errada: o comportamento é documentado, não transitório, e repetir só gasta chamadas. B está errada: abre mão da garantia estrutural do schema sem resolver a robustez da comparação. C está errada: contraria a documentação, que justamente ressalva a capitalização.
Fonte: https://platform.claude.com/docs/en/build-with-claude/structured-outputs

Questão 46 · D6 Prompt & Context Engineering · Context Engineering · média

Um pipeline gera resumos de contratos. O time quer registrar o rascunho, avaliar a revisão do rascunho contra critérios e, entre as etapas, decidir se segue para o refinamento ou desvia para revisão humana. Qual a melhor abordagem?

A. **Dividir em chamadas sequenciais (rascunho, revisão, refinamento) e ramificar a partir da saída de cada etapa.** [CORRETA]

B. Um único prompt que pede rascunho, revisão e versão final, retornando só a versão final para reduzir tokens.

C. Aumentar o effort da chamada única para que a revisão interna fique mais rigorosa e dispensar o log das etapas.

D. Confiar no adaptive thinking para fazer as três etapas internamente, já que o modelo raciocina sobre múltiplos passos sozinho.

Resposta correta: A. Correta (A): encadear prompts (prompt chaining) é útil quando você precisa inspecionar saídas intermediárias ou impor uma estrutura de pipeline; no padrão de autocorreção cada etapa é uma chamada separada, o que permite log, avaliação e ramificação. B esconde rascunho e revisão, impossibilitando o desvio. D está errada: o modelo pode raciocinar sobre vários passos, mas o raciocínio interno não oferece um ponto onde seu código decida o caminho. C não cria nenhum ponto de inspeção; effort maior não substitui o log das etapas.
Fonte: https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices

Questão 47 · D2 Applications & Integration · Software Engineering Foundations · fácil

Um chat web mostra a resposta do Claude token a token. O usuário envia cada pergunta por uma requisição POST normal, e o servidor só precisa empurrar texto de volta durante a geração. Qual transporte do servidor para o navegador é a melhor escolha?

A. Chamar a API da Anthropic direto do navegador com a chave embutida no JavaScript, eliminando o servidor intermediário

B. Polling a cada 100 ms em um endpoint, buscando os novos tokens acumulados

C. WebSocket, porque streaming de respostas exige um canal bidirecional entre navegador e servidor

D. **Server-Sent Events sobre HTTP, pois o fluxo é apenas do servidor para o cliente e o navegador reconecta automaticamente** [CORRETA]

Resposta correta: D. Correta (D): SSE é unidirecional, roda sobre HTTP e tem reconexão automática, o que basta quando a entrada do usuário vai por POST. C está errada porque a necessidade de bidirecionalidade não existe aqui; WebSocket só compensa quando o cliente também envia mensagens frequentes no mesmo canal. B está errada porque polling gera tráfego e latência desnecessários para um fluxo contínuo. A está errada porque expor a chave de API no navegador é uma falha de segurança grave.
Fonte: https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events

Questão 48 · D2 Applications & Integration · Configuration Management · fácil

Seu time quer que todos no repositório iniciem o Claude Code com o mesmo modelo por padrão, e que mudar essa escolha passe por code review e possa ser revertida. Qual abordagem?

A. Cada dev roda /model e salva a escolha, que é gravada no ~/.claude/settings.json de cada um.

B. Escrever no CLAUDE.md do projeto uma linha pedindo que Claude use o modelo desejado.

C. **Definir a chave model em .claude/settings.json e versioná-la no repositório.** [CORRETA]

D. Definir a chave model no .claude/settings.local.json de cada dev.

Resposta correta: C. Correta (C): o settings.json de projeto é compartilhado via git, então a escolha passa por PR e tem histórico para rollback. A fica no settings pessoal de cada pessoa, sem padronizar. B está errada: CLAUDE.md é contexto, não configuração, e não troca o modelo da sessão. D é arquivo pessoal, fora do controle de versão.
Fonte: https://code.claude.com/docs/en/settings

Questão 49 · D6 Prompt & Context Engineering · Prompt Engineering · fácil

A saída de Claude é enviada a um gateway de SMS que quebra com markdown. O system prompt diz 'NUNCA use markdown' e ainda aparecem asteriscos de vez em quando. Qual a melhor melhoria?

A. Mover a proibição para o fim de cada mensagem do usuário e tirá-la do system prompt.

B. Adicionar a regra de que erros de formatação serão penalizados e a resposta será rejeitada pelo gateway.

C. Escrever a mesma proibição três vezes e em letras maiúsculas para reforçar a prioridade da regra no prompt.

D. **Explicar o motivo: a resposta vai por SMS e o markdown aparece literalmente para o usuário.** [CORRETA]

Resposta correta: D. Correta (D): dar contexto ou motivação por trás da instrução ajuda Claude a entender seu objetivo e generalizar para casos não previstos, em vez de seguir uma proibição seca. C só aumenta a ênfase, sem informar o porquê. A troca o local da regra sem acrescentar informação. B é ameaça sem contexto; não diz a Claude qual é o problema real do canal SMS.
Fonte: https://platform.claude.com/docs/en/build-with-claude/prompt-engineering/claude-prompting-best-practices

Questão 50 · D7 Security & Safety · AI Application Security · difícil

Um agente que resume páginas web e e-mails de remetentes desconhecidos vai para produção. Quais três práticas são coerentes com a defesa contra injeção indireta? Selecione 3.

A. **Testar antes do deploy com documentos, e-mails e saídas de tools com tentativas de injeção deliberadas.** [CORRETA]

B. Inserir o texto das páginas no system prompt para que Claude dê mais peso ao conteúdo recuperado durante o resumo.

C. **Dar ao agente apenas as tools e dados necessários, em ambiente isolado com permissões restritas.** [CORRETA]

D. Dispensar testes adversariais, já que Claude é inerentemente resiliente a esse tipo de ataque e não precisa de validação.

E. **Analisar continuamente as saídas atrás de sinais de injeção bem-sucedida e refinar prompts e filtros.** [CORRETA]

Resposta correta: A, C, E. Corretas (C, A, E): red-team com conteúdo hostil antes do deploy, monitoramento contínuo e menor privilégio (sandbox, sem segredos desnecessários) formam camadas que limitam o dano mesmo se uma injeção passar. B está errada: conteúdo de terceiros não pertence ao system prompt; deve ir em tool_result, tratado como dado não confiável. D está errada: a resiliência do modelo não substitui as camadas adicionais nem os testes.
Fonte: https://platform.claude.com/docs/en/test-and-evaluate/strengthen-guardrails/mitigate-jailbreaks

Questão 51 · D2 Applications & Integration · Claude Application Design · fácil

Você quer testar um plugin (com skills e hooks) apenas neste repositório, sem afetar seus outros projetos nem os colegas que usam o mesmo repo. Qual escopo de instalação escolher?

A. User scope, que o habilita para você em todos os projetos desta máquina.

B. Escopo gerenciado, definido pela organização para todos os desenvolvedores.

C. Project scope, registrando-o no .claude/settings.json versionado e pedindo aos colegas que o ignorem.

D. **Local scope, que o habilita só para você neste repositório.** [CORRETA]

Resposta correta: D. Correta (D): local scope vale só para você neste repositório. A alcançaria todos os seus projetos. C habilita o plugin para todos que trabalham no repo via settings versionado, e pedir que ignorem não é um controle. B é política da organização, não uma escolha de teste pessoal.
Fonte: https://code.claude.com/docs/en/plugins/overview

Questão 52 · D1 Agents & Workflows · Agent Architecture · fácil

Um pipeline de três chamadas ao Claude extrai campos de uma fatura, concilia com o ERP e redige um e-mail de cobrança. Às vezes a extração devolve campos obrigatórios vazios e o e-mail final sai confiante, porém com dados errados. Qual a melhor abordagem?

A. Acrescentar ao prompt do e-mail a instrução "tenha cuidado se os dados estiverem incompletos" e manter o restante do fluxo como está.

B. Fundir as três etapas em um único prompt maior, para que o modelo enxergue a tarefa inteira de uma vez.

C. **Inserir um gate programático após a extração que valide os campos obrigatórios e barre ou repita a etapa.** [CORRETA]

D. Trocar o pipeline por um agente autônomo que decide sozinho, em tempo de execução, quais etapas executar e em que ordem.

Resposta correta: C. Correta (C): em prompt chaining, checagens programáticas (gates) entre etapas garantem que o processo continua no caminho certo; o erro é barrado onde nasce, de forma determinística. B remove os pontos de controle e piora a depuração. A depende de o modelo final perceber um dado ausente que ele nem recebe como erro. D troca um fluxo de passos conhecidos por autonomia, aumentando custo e imprevisibilidade sem atacar a causa.
Fonte: https://www.anthropic.com/engineering/building-effective-agents

Questão 53 · D1 Agents & Workflows · Agent Construction with Claude · difícil

Um agente construído com o Claude Agent SDK expõe uma ferramenta customizada get_order por meio de um servidor MCP em processo, registrado em mcpServers com a chave "shop". Claude enxerga e usa a ferramenta, mas, na execução sem interação humana, cada chamada fica parada esperando aprovação de permissão. Qual ajuste faz as chamadas rodarem sem prompt?

A. Marcar a ferramenta com readOnlyHint: true nas annotations, pois ferramentas somente leitura dispensam aprovação.

B. Reescrever a descrição da ferramenta para dizer que é segura, de modo que Claude não peça aprovação.

C. Passar tools: ["get_order"] nas opções, para a ferramenta ficar disponível e aprovada.

D. **Listar mcp__shop__get_order (ou o curinga mcp__shop__*) em allowedTools.** [CORRETA]

Resposta correta: D. Correta (D): ferramentas de um servidor MCP em processo aparecem como mcp__{servidor}__{ferramenta}, e listar esse nome (ou o curinga do servidor) em allowedTools as aprova sem prompt. C está errada: a opção tools filtra apenas as ferramentas built-in (ferramentas MCP não são afetadas) e não concede permissão. A está errada: readOnlyHint é metadado que só controla se a chamada pode ocorrer em paralelo com outras somente leitura; annotations não são aplicação de regra. B está errada: a descrição serve para Claude decidir quando chamar a ferramenta e não altera o fluxo de permissões.

Fonte: https://code.claude.com/docs/en/agent-sdk/custom-tools

Pontuação por domínio

Domínio	Questões	Acertos	%
D1 Agents & Workflows	8		
D2 Applications & Integration	17		
D3 Claude Code	2		
D4 Eval, Testing & Debugging	1		
D5 Model Selection & Optimization	9		
D6 Prompt & Context Engineering	6		
D7 Security & Safety	4		
D8 Tools & MCPs	6		
Total	**53**		

Referência: o exame real exige nota escalada 720 (escala 100–1000). Como estimativa grosseira, ~72% de acertos (≥ 39 de 53) indica nível de aprovação. Não é a conversão oficial, pois a nota é escalada.

Tópicos para revisar

Para cada questão errada, anote o domínio/skill (coluna 'Skill' do resumo) e releia a documentação oficial correspondente. Priorize os domínios de maior peso: Applications & Integration (33,1%) e Model Selection & Optimization (16,8%).

_____

_____

_____